

Types construits - Listes

Term NSI — Septembre 2026

Fiche d'exercices sur le cours python avancé, types construits. Algorithmes de somme, moyenne, recherche dans une liste, nombre d'occurrences.

1 Questions à reponses courtes

1.1 fonctions et méthodes

Les **fonctions** que l'on peut utiliser avec des listes, des tuples ou des chaînes str sont: `len` et `list`. Les **méthodes** associées aux listes sont : `append`, `remove`, `pop`, `index`, `extend`

1. Avec la liste `L = ["a", "b", "c", "e"]`, comment obtenir la longueur de la liste (le nombre d'éléments)? Quelle sera la valeur retournée?
2. Comment insérer le caractère `"d"` entre `"c"` et `"e"`?
3. Comment transformer la chaîne de caractères « abcde » en une liste `["a", "b", "c", "d", "e"]`?
4. Transformer cette liste `L` en `["b", "b", "c", "d", "e"]`
5. Comment supprimer le caractère « c » et obtenir ainsi `["b", "b", "d", "e"]`?
6. Quelle instruction permet d'ajouter `"f"` à la fin de la liste?
7. Comment obtenir l'indice du caractère `"d"` dans cette liste?
8. On dispose de la liste non triée `L = ["e", "c", "d", "b", "a"]`. Avec quelle instruction peut-on trier `L` de manière croissante `"a" -> "e"`?
9. Comment retirer le dernier élément de la liste `L`, et le placer dans la variable `dernier`?
10. Comment ajouter en une seule instruction les éléments de la liste `L2 = ["o", "p", "q", "r", "s"]` à `L`?

1.2 Découpage d'une Liste (slicing)

Le découpage de liste (*slicing*) permet d'extraire une séquence d'une liste. Soit `L` une liste. La syntaxe `L[i:j]` permet d'extraire tous les éléments consécutifs compris entre l'élément de rang `i` inclus et l'élément de rang `j` exclus (jusqu'à `j-1`).

On donne:

```
1 mois = ["Janvier", "Fevrier", "Mars", "Avril",
2         "Mai", "Juin", "Juillet", "Aout", "Septembre"
3         "Octobre", "Novembre", "Decembre"]
```

1. En utilisant la technique du *slicing* écrire une instruction permettant:
 - d'extraire de la liste `mois` uniquement les 6 premiers mois.
 - d'extraire les 2 mois d'été
2. Les objets créés, sont-ils copiés de la liste d'origine par *valeur* ou bien par *référence*?

1.3 Compréhension de liste

1. Quelle(s) expression(s) Python a-ont pour valeur la liste `[2, 5, 8, 11]`?
 - `[3 * i + 2 for i in range(3)]`
 - `[3 * i + 2 for i in range(4)]`
 - `[3 * i - 1 for i in range(1,5)]`
 - `[3 * i - 1 for i in range(4)]`

2. Créer une liste des 50 nombres pairs à->98 en compréhension de liste. (2 méthodes).

2 Traitement des éléments de liste

2.1 Additionner les valeurs

Le programme suivant ajoute les valeurs d'une liste dans une variable.

1. Compléter le script:

```
1 L = [1,2,3,4,5,6,7,8,9,10]
2 accumulateur = 0
3 for ... in ...
4     accumulateur += ...
5 print(...)
```

2. Ecrire une fonction `additionne` à partir de ce script.
3. Adapter le script de cette fonction pour qu'elle additionne les chiffres dans un mot, contenant uniquement des caractères compris entre « 0 » et « 9 » Exemple:

```
1 >>> additionne("01234")
2 10
```

3 Traitement d'une liste de notes

On souhaite faire un traitement python sur les notes obtenues au cours des 2 premiers trimestres. Créer une liste `notes` et mettre directement les valeurs 10, 8.9, 9.9, 12.3, 11.1

1. Quelle sera l'instruction correspondante?
2. Ajouter maintenant les notes obtenues lors de la période du 3e trimestre: 12.3, 13.1, 14.5, 20. Quelle-s instruction-s allez vous utiliser?

Somme

Le calcul de la somme des éléments nécessite d'utiliser une boucle bornée et accumuler les valeurs dans une variable, que l'on nomme par exemple, `s`.

Dans une fonction que vous appellerez `somme` : Définir une variable `s` que vous mettrez à 0. A l'aide d'une boucle bornée, parcourir les éléments de la liste, et ajouter chaque valeur à `s`. Retourner `s`.

3. Ecrire le script de cette fonction.

Vous définirez maintenant une fonction `moyenne`, qui prend pour paramètres `notes` (une liste de notes). La fonction calculera la moyenne, en ajoutant toutes les valeurs, et en divisant par le nombre de notes.

Aide: On utilisera la fonction `len(notes)` qui retourne le nombre d'éléments de `notes`. La fonction `moyenne` va alors retourner `s/len(tab)`

4. Ecrire le script de la fonction `moyenne`, et calculer la somme `s`, puis la moyenne des valeurs de `notes`

Maximum

Vous définirez une fonction `maximum`, qui prend pour paramètres `notes` (une liste de notes). On affecte à une variable `m` la première valeur de la liste. On parcourt ensuite les éléments `e`, un par un, à l'aide d'une boucle bornée:

- Si l'élément `e` est supérieur à `m`, on affecte à `m` la valeur `e`.
- sinon, on poursuit la boucle bornée.

A la fin de la boucle, on retourne la valeur `m`.

5. Ecrire le script de cette fonction `maximum`

Minimum

6. adapter le script précédent pour écrire une nouvelle fonction `minimum`, qui recherche le minimum dans la liste `notes`.

Indice du maximum

On cherche l'indice de la position de la valeur max dans une liste. On utilise le script de la fonction `maximum`, mais cette fois, la boucle bornée se fait sur les indices et non les valeurs de `notes`:

```
1  for i in range(len(notes)):
2  ...
```

On retourne l'indice de la plus grande valeur, `indice_max`

Exemple:

```
1  >>> indice_du_max(notes)
2  8
```

7. Ecrire le script de cette fonction

Nombre d'occurrences

On cherche à compter le nombre de notes supérieures ou égales à la moyenne 10. Pour cela, on utilise une variable `n` que l'on met à zéro en début de script. Puis à l'aide d'une boucle bornée, on parcourt les éléments de la liste `tab`. Chaque fois que la valeur de `e` est supérieure ou égale à 10.0, on incrémente `n` d'une unité. On retourne enfin la valeur `n`.

8. Ecrire le script de cette fonction.

4 Compteur d'espaces

Dans cet exercice, on considère des phrases composées de mots.

- On appelle « mot » une chaîne de caractères composée avec des caractères choisis parmi les 26 lettres minuscules ou majuscules de l'alphabet,
- On appelle *phrase* une chaîne de caractères :
 - ▶ composée avec un ou plusieurs *mots* séparés entre eux par un seul caractère espace ' ',
 - ▶ se finissant :
 - soit par un point '.' qui est alors collé au dernier mot,
 - soit par un point d'exclamation '!' ou d'interrogation '?' qui est alors séparé du dernier mot par un seul caractère espace ' '.

Voici deux exemples de phrases :

- “Cet exercice est simple.”
- “Le point d'exclamation est séparé !”

Après avoir remarqué le lien entre le nombre de mots et le nombre de caractères espace dans une phrase, programmer une **fonction** `nombre_de_mots` qui prend en paramètre une phrase et renvoie le nombre de mots présents dans cette phrase.

```
1 >>> nombre_de_mots('Cet exercice est simple.')
2 4
3 >>> nombre_de_mots('Le point d exclamation est séparé !')
4 6
5 >>> nombre_de_mots('Combien de mots y a t il dans cette phrase ?')
6 10
7 >>> nombre_de_mots('Fin.')
8 1
```

5 Parcours de chaîne de caractères, mot à trou

On considère des chaînes de caractères contenant uniquement des majuscules et des caractères *appelées mots à trous*.

Par exemple `INFO*MA*IQUE`, `***I***E**` et `*S*` sont des mots à trous.

Programmer une fonction `correspond` qui :

- prend en paramètres deux chaînes de caractères `mot` et `mot_a_trous` où `mot_a_trous` est un mot à trous comme indiqué ci-dessus,
- renvoie :
 - `True` si on peut obtenir `mot` en remplaçant convenablement les caractères `'*'` de `mot_a_trous`.
 - `False` sinon.

Exemple :

```
1 >>> correspond('INFORMATIQUE', 'INFO*MA*IQUE')
2 True
3 >>> correspond('AUTOMATIQUE', 'INFO*MA*IQUE')
4 False
5 >>> correspond('STOP', 'S*')
6 False
7 >>> correspond('AUTO', '*UT*')
8 True
```

6 Exercice de type bac – Statistiques sportives en EPS

Un professeur d'EPS enregistre les performances des élèves d'une classe en **saut en longueur**. Chaque élève réalise 3 essais, exprimés en mètres. Les données sont stockées dans une table (liste de listes) nommée `resultats` :

```
1 resultats = [['nom', 'record', 'essai1', 'essai2', 'essai3'],
2              ['Lina', 0, 3.85, 4.10, 3.95],
3              ['Karim', 0, 4.20, 4.05, 4.30],
4              ['Sofia', 0, 3.60, 3.75, 3.90],
5              ['Yanis', 0, 4.50, 4.40, 4.55]]
```

- La ligne d'indice `0` contient les en-têtes de colonnes.
- Chaque ligne suivante représente un élève : son nom, son record (colonne `1`, non encore calculé – d'où la valeur `0`), puis ses 3 essais (colonnes `2`, `3` et `4`).
- Pour un élève `eleve = resultats[i]`, la liste de ses essais est donc `eleve[2:]`.

Les réponses aux questions de compréhension sont à rédiger sur la copie. Les scripts à compléter sont à recopier avec la partie manquante remplie.

1. Donner la valeur de `resultats[2][3]`. À quoi correspond cette valeur ? Écrire l'instruction Python permettant d'obtenir la liste des trois essais de Karim.

2. On souhaite écrire une fonction `meilleure_perf` qui renvoie la meilleure performance (la plus grande distance) réalisée parmi une liste d'essais.

```
1 def meilleure_perf(essais):
2     perf = essais[...]
3     for e in essais:
4         if e ... perf:
5             perf = ...
6     return ...
```

2.a. Pourquoi initialise-t-on `perf` avec `essais[0]` plutôt qu'avec `0` ?

3. On souhaite maintenant calculer le record de chaque élève et le placer directement dans la table `resultats`, à la colonne d'indice `1`.

```
1 def enregistre_records(resultats):
2     for i in range(1, len(resultats)):
3         eleve = resultats[...]
4         eleve[...] = meilleure_perf(eleve[2:])
5
6     enregistre_records(resultats)
```

3.a. Cette fonction ne comporte pas d'instruction `return`. Pourtant, après l'appel `enregistre_records(resultats)`, la table `resultats` se trouve modifiée. Expliquer ce phénomène. Comment appelle-t-on ce type de fonctionnement ?

4. On souhaite écrire une fonction `record_classe` qui, une fois les records calculés, renvoie le **nom** de l'élève ayant réalisé la meilleure performance de la classe.

```
1 def record_classe(resultats):
2     meilleur_nom = resultats[1][...]
3     meilleure_valeur = resultats[1][...]
4     for i in range(..., len(resultats)):
5         eleve = resultats[i]
6         if eleve[1] ... meilleure_valeur:
7             meilleure_valeur = ...
8             meilleur_nom = ...
9     return meilleur_nom
10
11 print(record_classe(resultats))
```

4.a. Pourquoi la boucle commence-t-elle à l'indice 1 de la table `resultats` ?

5. Pour la finale du championnat, seuls les élèves ayant un record supérieur ou égal à un seuil fixé peuvent participer. On souhaite écrire une fonction `qualifies` qui renvoie une **nouvelle liste** contenant les noms des élèves qualifiés, sans modifier la table `resultats`.

```
1 def qualifies(resultats, seuil):
2     liste = ...
3     for i in range(1, len(resultats)):
4         eleve = resultats[i]
5         if eleve[1] ... seuil:
6             liste.append(...)
7     return liste
8
9 print(qualifies(resultats, 4.2))
```

5.a. Contrairement à la fonction `enregistre_records` de la question 3, la fonction `qualifies` ne modifie pas la table passée en paramètre : elle renvoie un nouveau résultat. Identifier, dans le code, l'instruction qui traduit cette différence de fonctionnement.